RECUPERACIÓN DE ESTRUCTURAS DE DATOS

Apellidos y nombres: Mamani Gutierrez Henry Antonio **Fecha:** 30/10/26
Asignatura: Estructura de Datos (2026) — Prof. Fred Torres Cruz **Modalidad:** Trabajo de Recuperación

Repositorio en GitHub: https://github.com/henrymamani-alt/henry-ejercicios

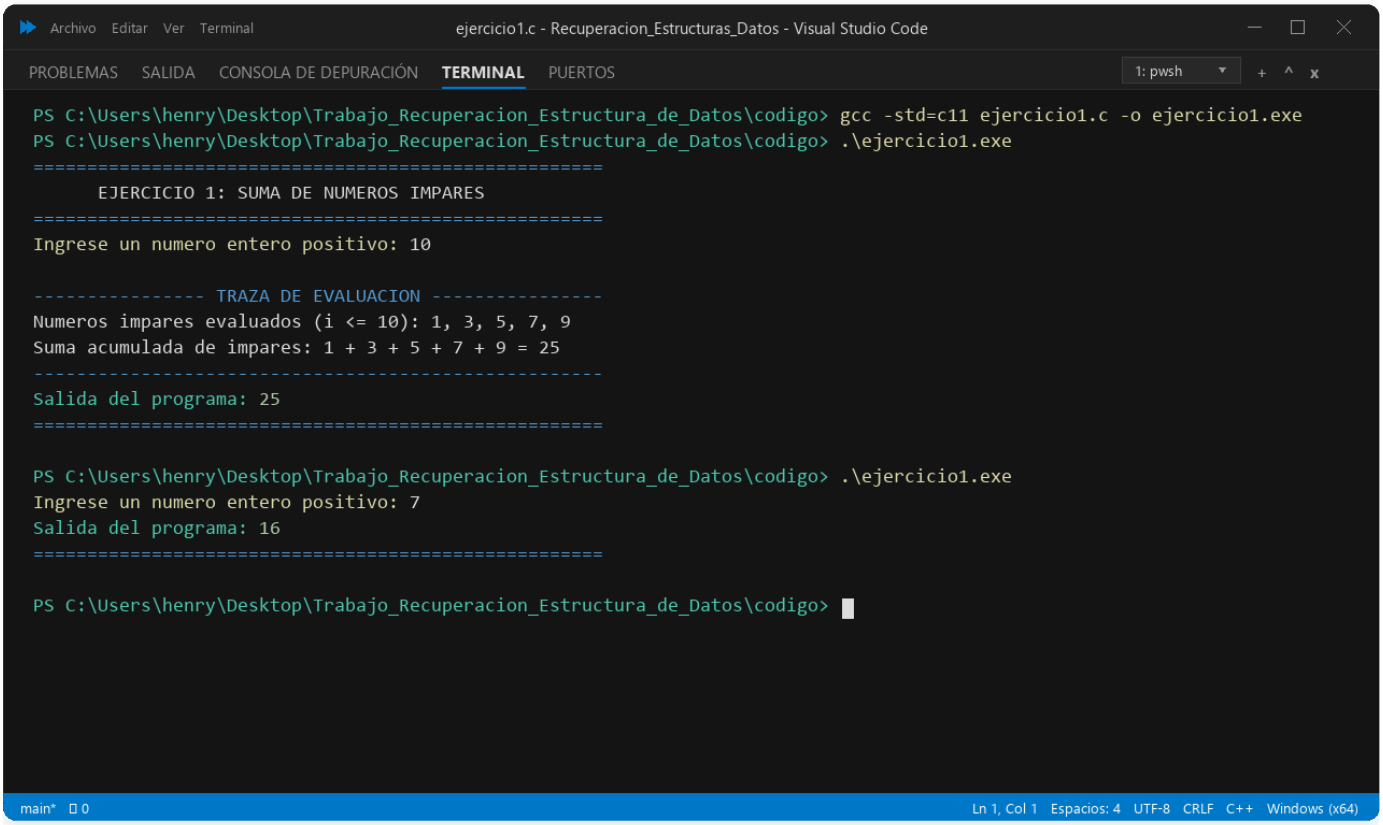
1. Suma de números impares.

Lea un entero positivo y muestre la suma de todos los números impares menores o iguales a él (ejemplo: 10 → 25).

```c
#include <stdio.h>

int main(void) {
    int n, i, suma = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i = i + 1) {
        if (i % 2 != 0) {
            suma = suma + i;
        }
    }
    printf("%d\n", suma);
    return 0;
}
```

Espacios completados en el código:
- **Espacio 1:** `1` — *Inicia en el primer número impar positivo.*
- **Espacio 2:** `i + 1` — *Paso de incremento secuencial de 1 en 1.*
- **Espacio 3:** `i % 2 != 0` — *Condición para verificar si el número es impar.*
- **Espacio 4:** `suma + i` — *Acumula el valor impar en la suma.*



2. Conteo de calles.

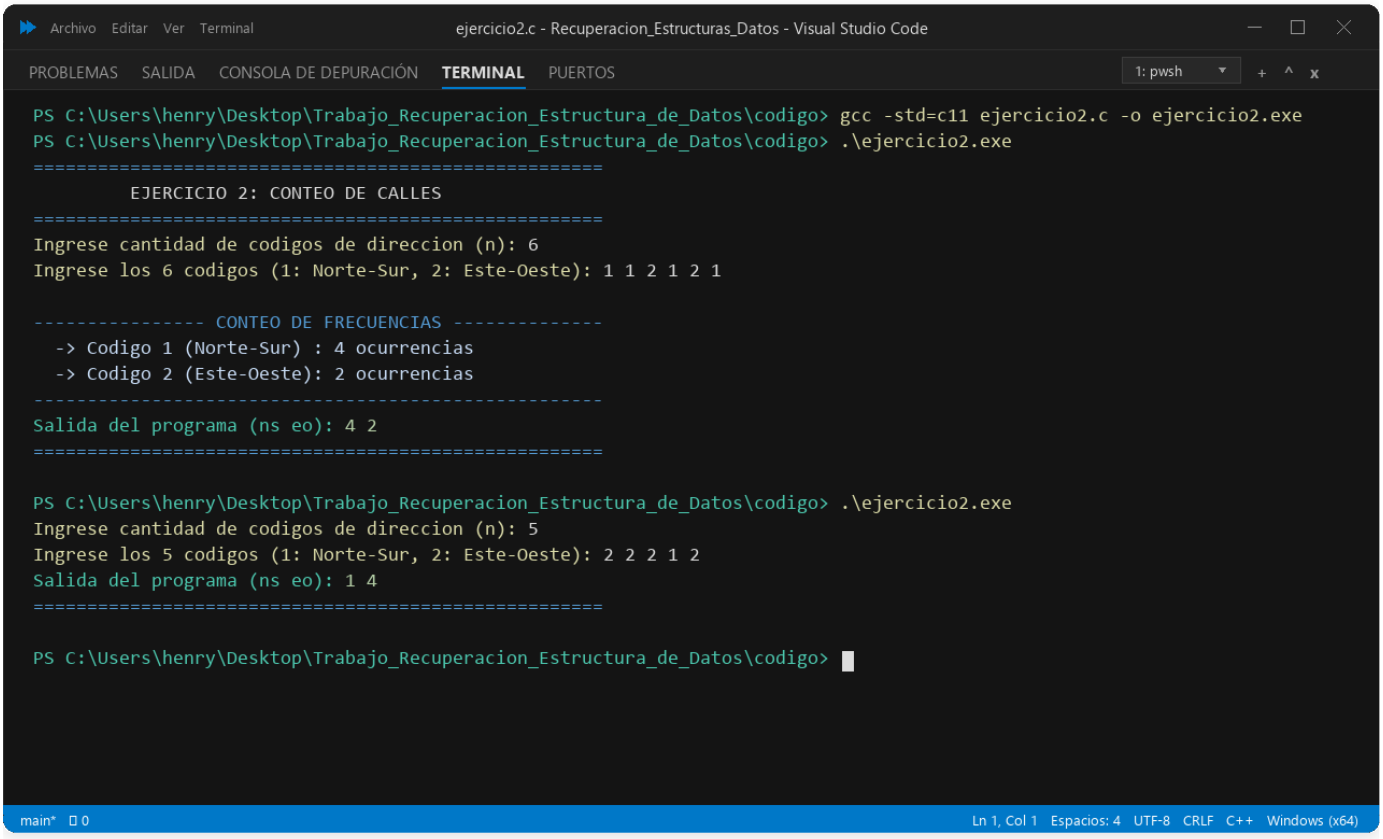
Lea n códigos de dirección: 1 representa Norte-Sur y 2 representa Este-Oeste. Imprima los totales respectivos (ejemplo: 1 1 2 1 2 1 → 4 2).

```c
#include <stdio.h>

int main(void) {
    int n, x, i, ns = 0, eo = 0;
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%d", &x);
        if (x == 1) {
            ns++;
        } else if (x == 2) {
            eo++;
        }
    }
    printf("%d %d\n", ns, eo);
    return 0;
}
```

Espacios completados en el código:

- **Espacio 1:** `i < n` — *Condición del for para leer los n códigos.*
- **Espacio 2:** `x == 1` — *Verifica si el código es 1 (Norte-Sur).*
- **Espacio 3:** `ns++` — *Incrementa el contador Norte-Sur.*
- **Espacio 4:** `eo++` — *Incrementa el contador Este-Oeste.*



3. Verificación de paréntesis con una pila.

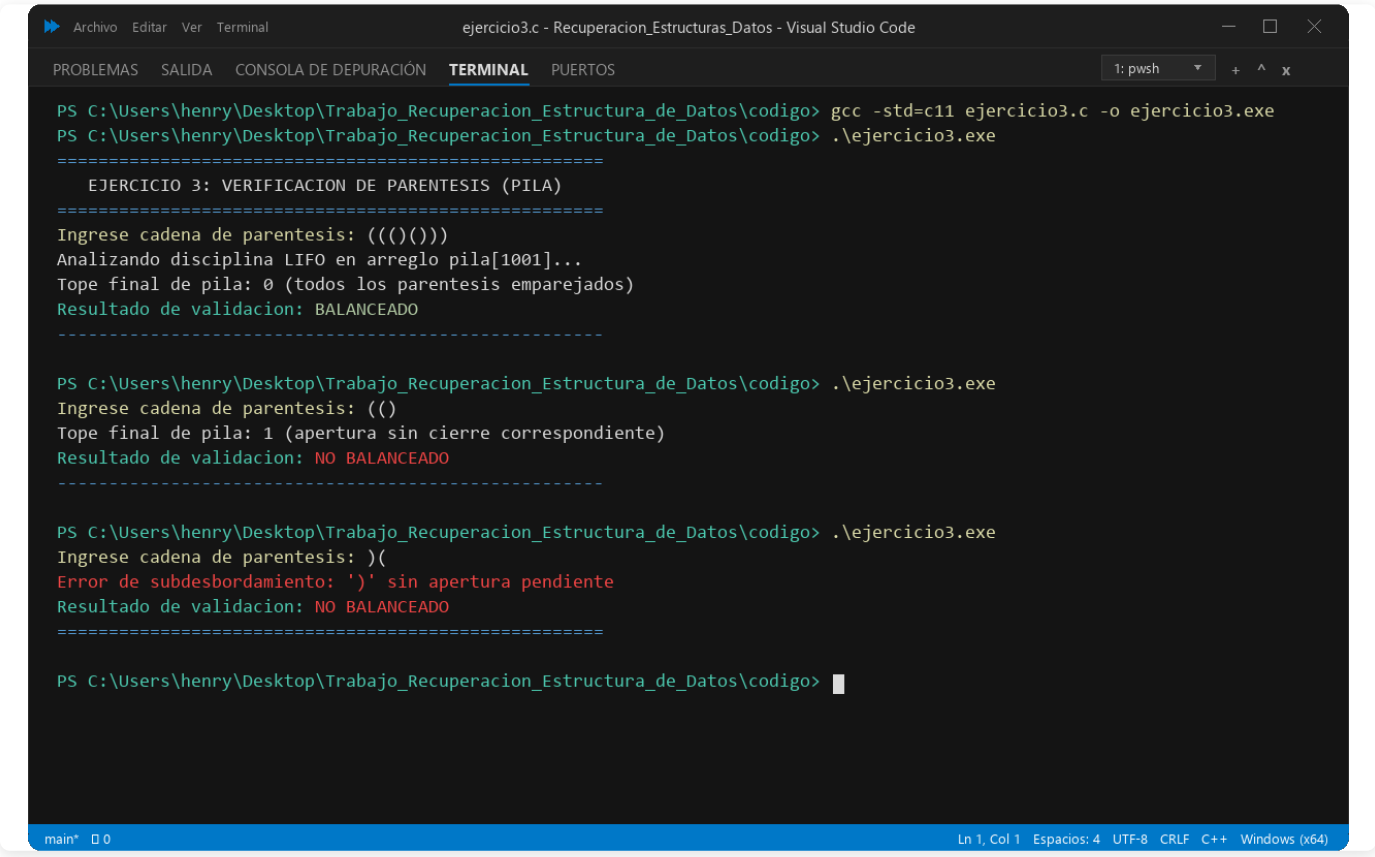
Lea una cadena compuesta solamente por (y). Escriba BALANCEADO si sus paréntesis están correctamente emparejados; en caso contrario, NO BALANCEADO. Complete cuatro espacios.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char s[1001], pila[1001];
    int i, tope = 0, valido = 1;
    scanf("%1000s", s);
    for (i = 0; s[i] != '\0'; i++) {
        if (s[i] == '(') {
            pila[ tope++ ] = '(';
        } else if (s[i] == ')') {
            if ( tope == 0 ) {
                valido = 0;
                break;
            }
            tope--;
        }
    }
    if ( valido && tope == 0 )
        puts("BALANCEADO");
    else
        puts("NO BALANCEADO");
    return 0;
}
```

Espacios completados en el código:

- **Espacio 1:** `tope++` — *Apila '(' en la posición tope y avanza el cursor.*
- **Espacio 2:** `tope == 0` — *Detecta si la pila está vacía al intentar cerrar (error).*
- **Espacio 3:** `tope--` — *Desapila el '(' emparejado decrementando el tope.*
- **Espacio 4:** `valido && tope == 0` — *Verifica que no hubo error y que la pila quedó vacía.*



4. Baúl con operaciones en ambos extremos.

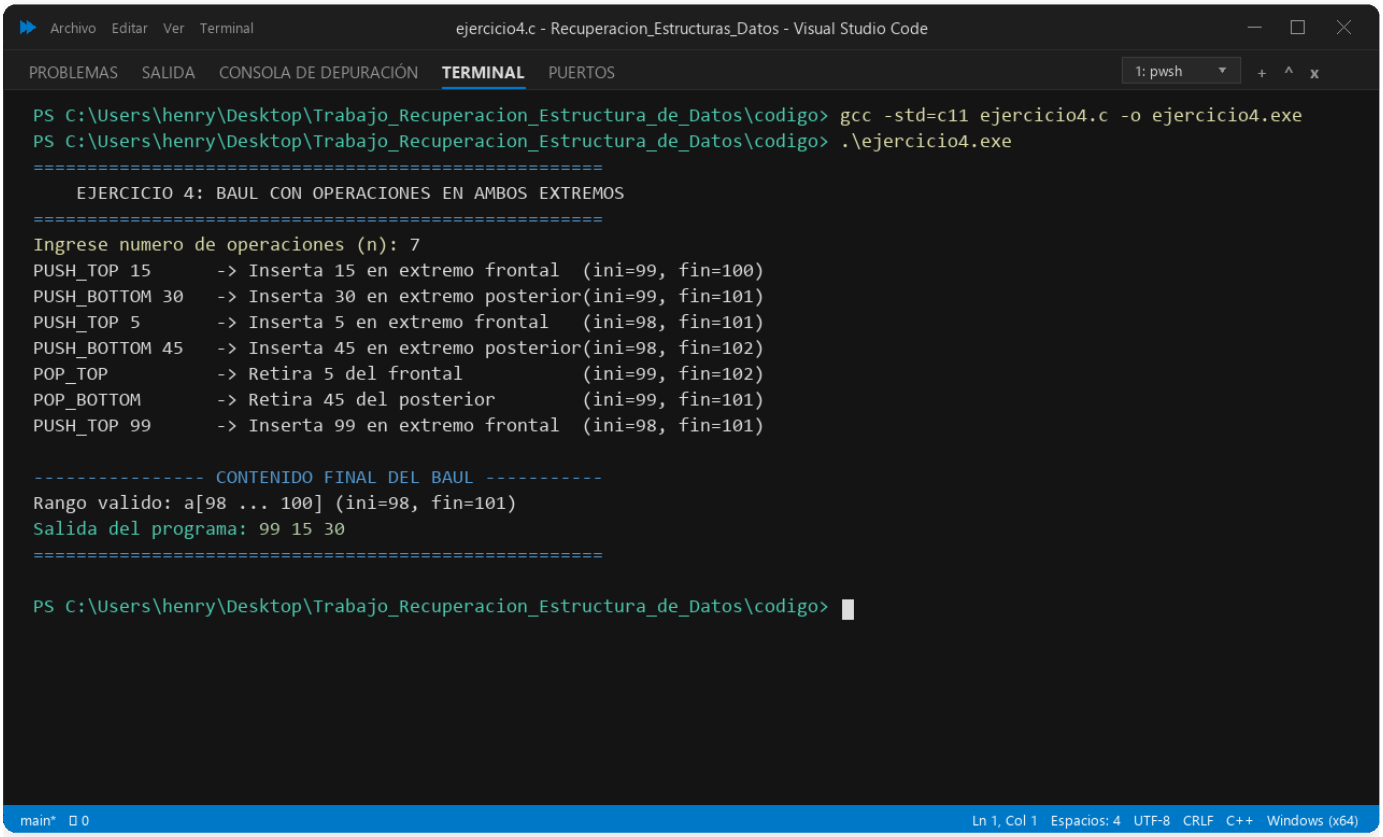
Use un arreglo como doble cola (deque). Las operaciones permiten insertar y retirar elementos por ambos extremos. Considere hasta 100 operaciones y valores enteros.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    int a[205], ini = 100, fin = 100, n, x, i, j;
    char op[20];
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        scanf("%19s", op);
        if (strcmp(op, "PUSH_TOP") == 0) {
            scanf("%d", &x); a[ --ini ] = x;
        } else if (strcmp(op, "PUSH_BOTTOM") == 0) {
            scanf("%d", &x); a[ fin++ ] = x;
        } else if (strcmp(op, "POP_TOP") == 0) {
            if (ini < fin) ini++ ;
        } else if (strcmp(op, "POP_BOTTOM") == 0) {
            if (ini < fin) fin-- ;
        }
    }
    if (ini == fin) puts("BAUL VACIO");
    else {
        for (j = ini; j < fin; j++) printf("%d ", a[j]);
        printf("\n");
    }
    return 0;
}
```

Espacios completados en el código:

- **Espacio 1:** `--ini` — *PUSH_TOP: decrementa ini e inserta en el extremo frontal.*
- **Espacio 2:** `fin++` — *PUSH_BOTTOM: inserta en el extremo posterior e incrementa fin.*
- **Espacio 3:** `ini++` — *POP_TOP: elimina del extremo frontal avanzando ini.*
- **Espacio 4:** `fin--` — *POP_BOTTOM: elimina del extremo posterior retrocediendo fin.*



5. Personas vetadas y recorrido inverso.

Lea exactamente diez nombres, excluya Santiago, Pedro, Ariadna, Marco, Juan y Angel. Imprima la cantidad admitida y sus nombres desde el último hasta el primero, utilizando un arreglo como pila.

```c
#include <stdio.h>
#include <string.h>

int main(void) {
    char nombre[10][40], pila[10][40];
    int i, tope = 0;
    for (i = 0; i < 10; i++) {
        scanf("%39s", nombre[i]);
        if (strcmp(nombre[i], "Santiago") != 0 &&
            strcmp(nombre[i], "Pedro") != 0 &&
            strcmp(nombre[i], "Ariadna") != 0 &&
            strcmp(nombre[i], "Marco") != 0 &&
            strcmp(nombre[i], "Juan") != 0 &&
            strcmp(nombre[i], "Angel") != 0 ) {
            strcpy(pila[ tope++ ], nombre[i]);
        }
    }
    printf("%d\n", tope );
    while (tope > 0) {
        puts(pila[ --tope ]);
    }
    return 0;
}
```

Espacios completados en el código:

- **Espacio 1:** `strcmp(nombre[i], "Angel") != 0` — *Excluye al sexto nombre vetado ('Angel').*
- **Espacio 2:** `tope++` — *Apila la persona admitida e incrementa tope.*
- **Espacio 3:** `tope` — *Imprime la cantidad de personas admitidas.*
- **Espacio 4:** `--tope` — *Desapila e imprime en orden inverso (LIFO).*

